

S3 et IAM approfondi

Stocker des objets, publier un site, donner une identité à une machine

CL-AWS1 — Jour 4 — Automatiser, superviser, sécuriser

Objectifs de la journée

À la fin de la journée, vous saurez :

- Expliquer ce qu'est le **stockage d'objets** et en quoi il diffère d'EBS (blocs) et d'un système de fichiers.
- Créer et manipuler des **buckets S3** en console et en CLI (`mb`, `cp`, `sync`, `ls`, `rm`).
- Choisir une **classe de stockage** selon le coût, la latence et la fréquence d'accès.
- Activer le **versioning** et récupérer une version antérieure d'un objet.
- Lire et **écrire une politique de bucket** (resource-based) et comprendre l'évaluation combinée avec les politiques IAM.
- Publier le **front StockLine** en site statique S3 — la seule exception publique qu'on s'autorise.
- Créer un **rôle IAM** et prouver qu'une EC2 peut lire un bucket **sans aucune clé configurée**.

Hier soir, nous avons détruit notre VPC pédagogique. Aujourd'hui, on travaille dans le **VPC par défaut** pour rester concentrés sur S3 et IAM.

Plan de la journée

1. **S3, le stockage d'objets** — buckets, clés, durabilité.
2. **S3 en pratique** — console, `aws s3` et `aws s3api`.
3. **Les classes de stockage** — du Standard au Deep Archive.
4. **Versioning et chiffrement** — se protéger de soi-même.
5. **Accès public, partage et site statique** — BPA, politiques de bucket, URL présignées, le front StockLine en ligne. 🖥️ Démon (acte A) + ✏️ Exercice 4.1.
6. **IAM approfondi : les rôles** — une identité sans clés pour EC2. 🖥️ Démon (acte B) + ✏️ Exercice 4.2.
7. Récap, quiz, teardown.

1. S3, le stockage d'objets

Simple Storage Service — le plus ancien service d'AWS (2006)

Blocs, fichiers, objets : trois façons de stocker

	Blocs (EBS)	Fichiers (NFS...)	Objets (S3)
Unité	secteurs bruts	fichier dans une arborescence	objet = données + clé + métadonnées

	Blocs (EBS)	Fichiers (NFS...)	Objets (S3)
Accès	attaché à une instance	monté par plusieurs machines	API HTTP (GET/PUT), de partout
Modification	en place, à l'octet	en place	remplacement complet de l'objet
Taille	volume fixe à provisionner	quota	illimitée , paiement à l'usage
Usage type	disque d'une EC2 (vu au J2)	partage d'équipe	sauvegardes, images, sites statiques, logs

À retenir : S3 n'est **pas un disque**. On n'y « ouvre » pas un fichier pour modifier 3 octets : on **envoie** et on **récupère** des objets entiers, via HTTP.

Le bucket : global par son nom, régional par ses données

Un **bucket** est le conteneur de vos objets. Deux règles souvent confondues :

- Le **nom** est unique **au niveau mondial** — tous comptes AWS confondus. Si un client à Singapour possède **stockline**, personne d'autre ne peut le créer.
- Mais le bucket **vit dans une région** : ses objets sont stockés physiquement là où vous l'avez créé (**eu-west-3** pour nous). Localisation des données = enjeu RGPD.

Contraintes de nommage : **3 à 63 caractères**, minuscules, chiffres, tirets ; commence par une lettre ou un chiffre ; pas de format d'adresse IP.

```
aws s3 mb s3://abc-stockline-front --region eu-west-3 --profile formation
```

D'où notre convention : préfixer par votre trigramme (**abc-...**) — cela limite aussi les collisions entre vous en salle.

Objets, clés et pseudo-dossiers

```

Service S3 (noms mondiaux)          Bucket créé dans eu-west-3
+-----+
| Bucket : abc-stockline-front      |
|                                   |
| Clé : index.html                  --> 1 objet |
| Clé : error.html                  --> 1 objet |
| Clé : css/style.css                --> 1 objet |
| Clé : images/logo.png            --> 1 objet |
|                                   |
| "css/" n'est PAS un dossier :     |
| c'est juste un PRÉFIXE dans la clé de l'objet |
+-----+

```

La console affiche des « dossiers » par confort, mais S3 est **plat** : un objet = une clé complète + un contenu + des métadonnées.

Durabilité 11 neufs ≠ disponibilité

- **Durabilité : 99,999999999 %** (11 neufs). C'est la probabilité de **ne pas perdre** un objet sur un an. S3 réplique automatiquement chaque objet sur **au moins 3 AZ** (sauf One Zone). Ordre de grandeur : avec 10 millions d'objets, on perd statistiquement 1 objet tous les 10 000 ans.
- **Disponibilité : 99,99 %** (Standard). C'est la probabilité que l'objet soit **accessible maintenant**. Une indisponibilité n'est pas une perte.

⚠ Ce que les 11 neufs ne couvrent **pas** : vos propres bêtises. Un `aws s3 rm --recursive` malheureux supprime durablement... avec 11 neufs de fiabilité. La parade s'appelle le **versioning** (section 4).

💰 Free tier : **5 Go** de Standard, 20 000 GET, 2 000 PUT par mois la première année. À notre échelle, S3 est quasi gratuit — ce sont les **requêtes** et les **versions accumulées** qui finissent par coûter.

2. S3 en pratique

Console pour voir, CLI pour travailler

Les commandes du quotidien

Toujours avec votre profil du J1 (`--profile formation`, région `eu-west-3`) :

```

export AWS_PROFILE=formation
export AWS_DEFAULT_REGION=eu-west-3

aws s3 mb s3://abc-stockline-front      # make bucket
aws s3 cp index.html s3://abc-stockline-front/  # envoyer 1 fichier
aws s3 cp s3://abc-stockline-front/index.html .  # récupérer 1 fichier
aws s3 ls                                # lister MES buckets

```

```
aws s3 ls s3://abc-stockline-front/      # lister les objets
aws s3 rm s3://abc-stockline-front/ --recursive  # tout supprimer
aws s3 rb s3://abc-stockline-front      # remove bucket (doit être vide)
```

`cp` accepte `--recursive` pour un dossier entier. Mais pour synchroniser, il y a mieux...

aws s3 sync : le rsync du cloud

Au bloc Linux, vous avez utilisé `rsync` pour ne transférer **que les différences**. `sync` fait exactement pareil entre un dossier local et un bucket (dans les deux sens) :

```
aws s3 sync ./front s3://abc-stockline-front/      # local -> bucket
aws s3 sync s3://abc-stockline-front/ ./sauvegarde  # bucket -> local
aws s3 sync ./front s3://abc-stockline-front/ --delete  # miroir strict
aws s3 sync ./front s3://abc-stockline-front/ --dryrun  # simuler d'abord
```

- Comparaison sur **taille + date de modification** : seuls les fichiers changés partent.
- `--delete` supprime côté cible ce qui n'existe plus côté source — comme `rsync --delete`. À utiliser avec `--dryrun` d'abord, réflexe du bloc Linux.
- C'est LA commande de déploiement d'un site statique : on modifie, on `sync`, c'est en ligne.

aws s3 vs aws s3api : deux étages de la même fusée

	aws s3 (haut niveau)	aws s3api (bas niveau)
Philosophie	commandes « confort »	1 commande = 1 appel API
Exemples	<code>cp</code> , <code>sync</code> , <code>mb</code> , <code>ls</code>	<code>put-object</code> , <code>list-object-versions</code> , <code>put-bucket-policy</code>
Multipart, récursif	géré automatiquement	à vous de faire
Quand ?	95 % du quotidien	réglages fins : versioning, politiques, BPA...

```
aws s3 cp gros-fichier.zip s3://abc-bucket/      # découpe multipart toute seule
aws s3api put-object --bucket abc-bucket --key f.zip --body gros-fichier.zip
```

Aujourd'hui vous utiliserez les deux : `s3` pour les fichiers, `s3api` pour le versioning, les politiques et le Block Public Access.

3. Les classes de stockage

Payer le juste prix selon la fréquence d'accès

Les classes « chaudes »

Prix indicatifs eu-west-3, par Go et par mois — l'ordre de grandeur compte plus que le centime :

Classe	Stockage	Récupération	Durée min.	Cas d'usage
Standard	~0,024 \$	gratuite, immédiate	aucune	données actives (notre front)
Standard-IA	~0,013 \$	payante (~0,01 \$/Go), immédiate	30 jours	sauvegardes mensuelles
One Zone-IA	~0,010 \$	payante, immédiate	30 jours	données recréables (1 seule AZ !)

- **IA** = *Infrequent Access* : moitié prix au stockage, mais chaque lecture se paie.
- **One Zone-IA** renonce à la réplication multi-AZ : si l'AZ brûle, les données sont perdues. Réservé à ce qu'on peut régénérer (miniatures, caches).
- « Durée minimale » : supprimer un objet IA avant 30 jours → facturé 30 jours quand même.

Les classes « froides » et le pilote automatique

Classe	Stockage	Restitution	Durée min.	Cas d'usage
Glacier Instant Retrieval	~0,005 \$	immédiate, payante	90 jours	archives consultées ~1 × /trimestre
Glacier Flexible Retrieval	~0,004 \$	minutes à 12 h	90 jours	archives rarement lues
Glacier Deep Archive	~0,002 \$	jusqu'à 12-48 h	180 jours	conformité, rétention 7-10 ans
Intelligent-Tiering	variable	immédiate	aucune	fréquence d'accès imprévisible

- Flexible et Deep Archive : l'objet n'est **pas lisible immédiatement** — il faut demander une *restauration* et attendre.
- **Intelligent-Tiering** déplace automatiquement l'objet entre paliers selon l'usage réel, contre des frais de surveillance minimales. Le bon choix quand on ne sait pas.
- La transition automatique entre classes se configure par **règles de cycle de vie** (aperçu en démo de console, approfondi au J8 côté coûts).



Ce qui coûte vraiment sur S3

À notre échelle de formation, le stockage est négligeable (5 Go free tier). Les vraies lignes de facture, en production :

- **Les requêtes** : chaque GET/PUT/LIST se compte. Un site très visité sans cache devant = des millions de GET. (Teaser : CloudFront, J7.)
- **Le transfert sortant** vers Internet (~0,09 \$/Go après 100 Go/mois). Entrant : gratuit.
- **Les frais de récupération** des classes IA/Glacier — une classe froide mal choisie coûte plus cher que Standard.
- **Les durées minimales facturées** : 30/90/180 jours selon la classe.
- **Les versions accumulées** : avec le versioning, chaque version est un objet facturé. Un bucket « de 2 Go » peut en stocker 40.

Réflexe d'admin : le coût S3 se **supervise** (métriques de taille et de requêtes — J8) et se **nettoie** (cycle de vie).

4. Versioning et chiffrement

Se protéger des suppressions... et cocher la case sécurité

Le versioning : activation simple, retour impossible

Le versioning conserve **toutes les versions** de chaque objet. Chaque **PUT** sur une clé existante crée une nouvelle version, l'ancienne reste récupérable.

```
aws s3api put-bucket-versioning --bucket abc-stockline-front \  
--versioning-configuration Status=Enabled  
  
aws s3api list-object-versions --bucket abc-stockline-front \  
--prefix index.html
```

-  **Irréversible** : un bucket versionné ne redevient jamais « non versionné ». On peut seulement **suspendre** (**Status=Suspended**) — les versions existantes restent.
- Chaque version a un **VersionId** unique ; la plus récente est marquée **IsLatest: true**.
-  Chaque version est facturée comme un objet à part entière : versioning sans règle de cycle de vie = facture qui gonfle en silence.

Delete markers : la suppression qui n'en est pas une

Sur un bucket versionné, **aws s3 rm** ne détruit rien : il pose un **delete marker** (marqueur de suppression) qui devient la « version courante ».

- L'objet **disparaît des listings** et des GET (erreur 404)... mais toutes ses versions sont toujours là, et toujours facturées.
- **Restaurer** = supprimer le delete marker, ou recopier une ancienne version au sommet :

```
aws s3api copy-object --bucket abc-stockline-front \  
--copy-source "abc-stockline-front/index.html?versionId=ANCIEN_ID" \  
--replace
```

```
--key index.html
```

- **Détruire vraiment** un objet : supprimer chaque version par son **VersionId**. Conséquence pratique pour le teardown : un bucket versionné ne se vide pas avec un simple **rm --recursive** — il faut purger versions **et** delete markers (vu en démo).

Chiffrement au repos : survol

Depuis janvier 2023, **tout objet envoyé sur S3 est chiffré au repos**, sans rien configurer. La question n'est plus « chiffrer ? » mais « **qui gère la clé ?** » :

	SSE-S3 (défaut)	SSE-KMS
Clé gérée par	S3, invisible pour vous	AWS KMS : clé visible, auditable
Contrôle	aucun réglage	qui peut utiliser la clé, rotation, journal CloudTrail
Coût	inclus	💰 chaque appel KMS est facturé (~0,03 /10000) + 1 /mois/clé

- SSE-KMS s'impose quand la conformité exige de **prouver** qui a accédé aux clés, ou de séparer les droits « lire l'objet » et « utiliser la clé ».
- Piège classique : SSE-KMS sur un bucket à fort trafic → la facture KMS dépasse la facture S3.
- Pour StockLine aujourd'hui : SSE-S3 par défaut, on ne touche à rien.

5. Accès public, partage et site statique

Qui a le droit de toucher mes objets — et comment publier sans tout ouvrir ?

Block Public Access : votre meilleur ami

Par défaut, **tout bucket est privé** ET verrouillé par le **Block Public Access** (BPA) — 4 interrupteurs, tous activés à la création :

Case	Effet
BlockPublicAcls	rejette tout envoi d'ACL publique
IgnorePublicAcls	ignore les ACL publiques existantes
BlockPublicPolicy	rejette toute bucket policy publique
RestrictPublicBuckets	neutralise une policy publique déjà en place

- BPA existe à **deux niveaux** : le compte (global) et chaque bucket. Le plus restrictif gagne : un BPA compte activé rend toute publication impossible, même avec une policy parfaite.
- Pourquoi « meilleur ami » ? La quasi-totalité des fuites de données S3 de la décennie 2010 = buckets publics par erreur. BPA a été créé pour ça.
- **La seule exception qu'on s'autorise aujourd'hui** : le bucket du site statique StockLine — désactivation **sur CE bucket uniquement**, jamais au niveau du compte.

Identity-based vs resource-based : où est écrit le droit ?

Au J1, vous avez écrit des politiques **attachées à des identités** (utilisateurs, groupes). Il existe une seconde famille, attachée **aux ressources** :

	Politique IAM (identity-based)	Politique de bucket (resource-based)
Attachée à	utilisateur, groupe, rôle	le bucket lui-même
Répond à	« que peut faire cette identité ? »	« qui peut toucher ce bucket ? »
Champ Principal	absent (implicite : le porteur)	obligatoire — LA grande différence
Peut viser des anonymes	non	oui (" Principal ": "*")

C'est le **Principal** qui permet à un bucket d'autoriser un autre compte, un rôle précis... ou tout Internet (site statique). Une identity policy n'en a pas besoin : elle sait déjà à qui elle est attachée.

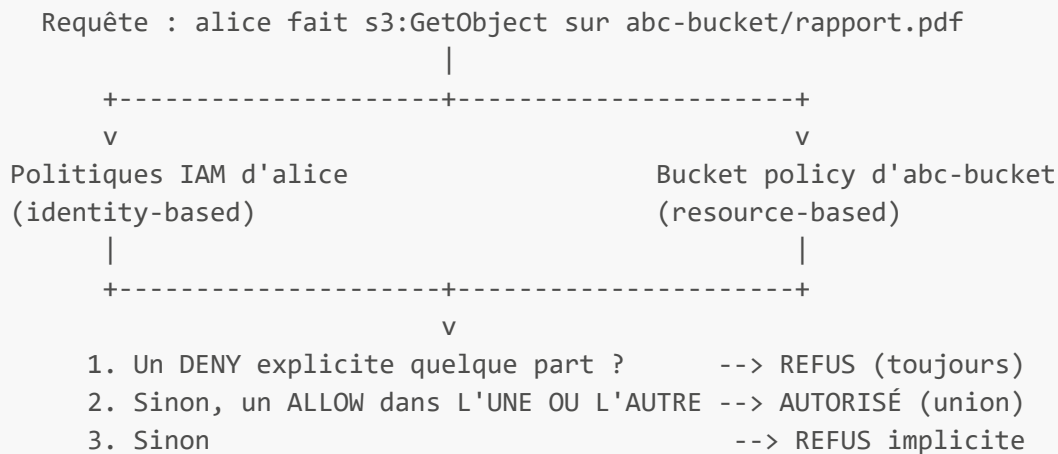
Lire une politique de bucket

Celle que nous poserons en démo (fichier `code/06-cl-aws1/politiques/bucket-policy-site-statique.json`, motif **PREFIX** remplacé par **sed**) :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LectureAnonymeDuSiteStatique",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::PREFIX-stockline-front/*"
    }
  ]
}
```

- **Principal**: "*" : tout le monde, y compris non authentifié — c'est ce qui rend le site public.
- **Resource**: `.../*` : **les objets** du bucket. Sans `/*`, l'ARN désigne le bucket lui-même (actions `s3:ListBucket`), pas son contenu. Erreur classique n° 1.
- **Action**: `s3:GetObject` uniquement : lecture. Personne ne peut lister, écrire ni supprimer.

L'évaluation combinée : union des Allow, le Deny gagne toujours



Même compte : un Allow d'un seul côté suffit. Un **Deny** explicite écrase tout, où qu'il soit écrit — c'est ce qui rend les « garde-fous » (deny non-TLS, deny delete) si puissants.

URL présignées : partager UN objet privé, temporairement

Besoin fréquent : envoyer un fichier privé à quelqu'un **sans** le rendre public, **sans** lui créer un compte AWS.

```
aws s3 presign s3://abc-stockline-data/exports/stock-2026-07.csv \
--expires-in 300
```

- Résultat : une URL HTTPS contenant une **signature** — quiconque la possède peut faire un GET, jusqu'à expiration (ici 300 s ; défaut 3600 s ; max 7 jours).
- L'URL agit **avec les permissions du signataire** : si VOUS ne pouvez pas lire l'objet, l'URL ne le pourra pas non plus.
- Le bucket reste privé, BPA reste activé : rien à ouvrir.
- Après expiration : **403 – Request has expired**.
- Cas d'usage : facture client, export ponctuel, livraison de build ; et côté applicatif, c'est ainsi qu'une API FastAPI ferait télécharger des fichiers à ses utilisateurs.

Héberger un site statique sur S3

S3 sait servir directement du HTML/CSS/JS : idéal pour le **front StockLine** (pas de serveur, pas d'instance à administrer, coût quasi nul).

- Activation : définir un **index document** (`index.html`) et un **error document** (`error.html`).
- Le site est servi sur le **website endpoint**, distinct de l'endpoint API :

```
http://abc-stockline-front.s3-website.eu-west-3.amazonaws.com
```

-  **HTTP seulement** sur cet endpoint — pas de HTTPS natif.
- L'error document s'affiche pour toute clé inexistante (404) : indispensable pour ne pas exposer d'erreurs XML brutes aux visiteurs.

Teaser : « en prod, on mettra **CloudFront** + **HTTPS** devant ce bucket — c'est le programme du J7. »

La recette complète du site statique

Trois verrous à ouvrir, dans l'ordre, **sur CE bucket uniquement** :

```
# 1. Désactiver le Block Public Access du bucket (pas du compte !)  
aws s3api put-public-access-block --bucket abc-stockline-front \  
  --public-access-block-configuration BlockPublicAcls=false,\  
  IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=false  
  
# 2. Poser la bucket policy de lecture publique (JSON vu plus haut)  
aws s3api put-bucket-policy --bucket abc-stockline-front \  
  --policy file://bucket-policy.json  
  
# 3. Activer l'hébergement web  
aws s3 website s3://abc-stockline-front \  
  --index-document index.html --error-document error.html
```

Sans l'étape 1, l'étape 2 échoue (**AccessDenied** : BPA bloque les politiques publiques). Sans l'étape 2, le site répond 403. C'est **le seul bucket public** de toute la formation.



Démo 4.1 — Acte A : « le front en ligne »

Fichier : `demos/06-cl-aws1/demo-4-1-s3-site-statique.md` (acte A, ~40 min)

Au programme, en direct :

1. Création du **front minimal StockLine** (`index.html` + `error.html`).
2. `mb`, `cp`, puis `sync` — et le cas « nom de bucket déjà pris ».
3. **Versioning** : activation, modification, `list-object-versions`, restauration.
4. Accès anonyme → **403** : BPA fait son travail.
5. Ouverture contrôlée : BPA du bucket, bucket policy (via `sed`), website hosting.
6. `curl` du website endpoint → **le front StockLine s'affiche**.
7. Bonus : **URL présignée** de 300 s sur un bucket privé, et son expiration.

Suivez avec votre trigramme : chaque commande est rejouable telle quelle.



Exercice 4.1 — Lire et écrire des politiques de bucket

Fichier : `exercices/06-cl-aws1/exercice-4-1-politique-bucket.md`

- **Durée : 45 min** — Difficulté : 3/5

- **Partie 1 — Lecture** : une bucket policy multi-statements vous est fournie (lecture publique partielle, deny non-TLS, écriture réservée à un rôle). Répondez précisément : *qui peut quoi ?*
- **Partie 2 — Écriture** : rédigez la politique du bucket `$PREFIX-partage-clients` à partir de 4 exigences métier, puis validez-la avec `aws s3api put-bucket-policy` et des tests `curl`.
- **Bonus** : restreindre par adresse IP source (`Condition` + `aws:SourceIp`).

Objectif : que le JSON d'une politique devienne aussi lisible pour vous qu'un `if` Python.

6. IAM approfondi : les rôles

Une identité sans clés — le vrai visage de la sécurité AWS

Le rôle : une identité qui ne possède aucune clé

Au J1, votre utilisateur IAM a reçu des **clés d'accès permanentes** (`aws configure`). Un **rôle** est une identité IAM d'un autre genre :

- **Aucun mot de passe, aucune clé d'accès permanente**. On ne se « connecte » pas à un rôle : on **l'endosse** (*assume*).
- Quand une entité endosse un rôle, **STS** (Security Token Service) lui délivre des **credentials temporaires** : `AccessKeyId` + `SecretAccessKey` + **token de session** + date d'**expiration** (quelques heures).
- Expirés ? STS en redélivre automatiquement. Volés ? Ils meurent tout seuls.
- Qui peut endosser un rôle ? Un service AWS (EC2, Lambda...), un autre compte, un utilisateur — c'est écrit dans sa **trust policy**.

Le rôle remplace le `aws configure` que vous avez fait au J1 — mais pour des **machines**, et sans rien stocker.

Pourquoi JAMAIS de clés d'accès sur une EC2

La tentation : faire `aws configure` dans l'instance avec ses clés du J1. **Interdit à vie**. Voici pourquoi :

- Les clés seraient **en clair** dans `~/.aws/credentials` : lisibles par toute personne (ou tout code) qui accède à l'instance, présentes dans les snapshots et les AML.
- Ce sont des clés **permanentes** : volées un mardi, encore valides à Noël.
- Elles portent **vos** droits d'humain, pas les droits minimaux de la machine.
- Les rotations deviennent un cauchemar : combien d'instances à mettre à jour ?

La bonne pratique AWS (et une question d'examen certifiée) : **un rôle IAM attaché à l'instance**. Credentials temporaires, rotation automatique, révocation immédiate en détachant le rôle, moindre privilège par machine. C'est ce qu'on prouve en démo, acte B.

Trust policy vs permissions policy : les deux faces d'un rôle

Un rôle porte **deux documents JSON** qu'il ne faut jamais confondre :

	Trust policy (politique d'approbation)	Permissions policy
Question	QUI peut endosser ce rôle ?	QUE peut faire ce rôle ?
Contient	Principal + <code>sts:AssumeRole</code>	des Action/Resource classiques
Notre cas	<code>"Service": "ec2.amazonaws.com"</code>	<code>s3:ListBucket</code> + <code>s3:GetObject</code> sur le bucket front

Fichiers fournis dans `code/06-cl-aws1/politiques/` : `trust-policy-ec2.json` et `policy-s3-lecture-front.json` (motif `PREFIX` remplacé par `sed`).

Moyen mnémotechnique : la trust policy est le **videur** (qui entre ?), la permissions policy est le **règlement intérieur** (que fait-on une fois entré ?).

Le flux complet : EC2 → rôle → STS → S3

```

+-----+ 1. l'instance porte un INSTANCE PROFILE
| EC2 t3.micro | (l'adaptateur qui fixe le rôle à l'EC2)
| instance profile: |
| abc-profile-ec2-s3|----- 2. endosse le rôle (trust policy: OK,
+-----+ ec2.amazonaws.com est approuvé)
|
| v
|
| +-----+
| 3. STS délivre des | Rôle |
| <-----| abc-role-ec2-s3 |
| credentials |
| TEMPORAIRES (clé + token + expiration,
| visibles via IMDSv2, renouvelés seuls)
|
| v
4. aws s3 ls s3://abc-stockline-front --> signé avec ces credentials
5. S3 évalue la PERMISSIONS POLICY --> lecture OK, écriture REFUSÉE

```

L'**instance profile** est le maillon souvent oublié : la console le crée en douce, la CLI vous oblige à le créer explicitement — tant mieux, vous comprendrez ce qui se passe.

Gérée vs inline — et le Policy Simulator pour tester sans exécuter

Où écrire la permissions policy d'un rôle ?

	Gérée (<i>managed</i>)	Inline
Existence	objet IAM autonome, avec son ARN	collée à une seule identité
Réutilisable / versionnée	oui : N identités, 5 versions, rollback	non — meurt avec son identité

	Gérée (<i>managed</i>)	Inline
Familles	gérées AWS (souvent trop larges : <code>AmazonS3ReadOnlyAccess</code> lit TOUS vos buckets) et gérées client (la norme en entreprise)	politique vraiment spécifique à UNE identité (notre démo : <code>put-role-policy</code>)

Et pour vérifier un droit **sans rien exécuter** : l'**IAM Policy Simulator** (<https://policysim.aws.amazon.com>).

- On choisit l'identité (le rôle), le service (S3), les actions, l'ARN de la ressource → verdict **allowed / denied**, avec la politique responsable.
- En CLI : `aws iam simulate-principal-policy --policy-source-arn <arn-du-rôle> --action-names s3:GetObject --resource-arns <arn-objet>`
- Il rejoue le moteur d'évaluation à blanc (union des allow, deny gagne). Réflexe pro : simuler **avant** de déployer — vous le pratiquerez dans l'exercice 4.2.



Démo 4.1 — Acte B : « le duo EC2 + S3 »

Fichier : `demos/06-cl-aws1/demo-4-1-s3-site-statique.md` (acte B, ~35 min)

Le moment « déclic » de la journée :

1. Création du rôle `abc-role-ec2-s3` : trust policy + permissions policy (`sed` sur les JSON fournis).
2. `create-instance-profile` + `add-role-to-instance-profile`.
3. Lancement d'une **t3.micro avec --iam-instance-profile** (VPC par défaut).
4. En SSH : `aws s3 ls` fonctionne **sans aucun aws configure**.
5. Tentative d'écriture → `AccessDenied` : le moindre privilège, prouvé.
6. Les credentials temporaires **vus de l'intérieur** via IMDSv2 (souvenez-vous du J2).
7. Vérification au **Policy Simulator**, puis **teardown complet**.



Exercice 4.2 — Un rôle en lecture seule sur un préfixe

Fichier : `exercices/06-cl-aws1/exercice-4-2-role-ec2-s3.md`

- **Durée : 60 min** — Difficulté : 3/5
- Créez un bucket privé `$PREFIX-exo42-logs` contenant 3 fichiers, dont certains sous le préfixe `app1/`.
- Créez un rôle **lecture seule, limité au préfixe `app1/*` uniquement**.
- Vérifiez vos permissions **au Policy Simulator AVANT tout lancement**.
- Attachez le rôle à une t3.micro et prouvez depuis l'instance : `app1/` se lit, tout le reste renvoie `AccessDenied`.
- **Teardown complet** exigé (instance, profile, rôle, bucket).
- **Bonus** : la même vérification en CLI avec `aws iam simulate-principal-policy`.

Récap de la journée

- **S3 = objets** (pas des blocs, pas des fichiers) : bucket au nom **mondialement unique** mais vivant **dans une région** ; clés plates, pseudo-dossiers ; **durabilité 11 neufs ≠ disponibilité**.
- **CLI** : `aws s3` (confort : `cp`, `sync` — le rsync du cloud) vs `aws s3api` (1 commande = 1 appel API).

- **Classes de stockage** : payer selon la fréquence d'accès ; gare aux frais de récupération et aux durées minimales.
 - **Versioning** : irréversible (suspendre seulement), delete markers, chaque version facturée.
 - **Chiffrement** : SSE-S3 par défaut partout ; SSE-KMS quand il faut contrôler la clé (💰 par appel).
 - **BPA** activé par défaut, compte + bucket : votre meilleur ami. Une seule exception aujourd'hui : le site statique.
 - **Bucket policy** = resource-based avec **Principal** ; évaluation : **union des Allow, un Deny gagne toujours**.
 - **URL présignée** : partager un objet privé, temporairement, avec les droits du signataire.
 - **Rôle IAM** : identité **sans clés**, credentials temporaires STS ; trust policy (qui endosse) ≠ permissions policy (que faire) ; instance profile ; **jamais de clés d'accès sur une EC2**.
-

Quiz — questions 1 et 2

Question 1 — Vous exécutez `aws s3 mb s3://stockline` et obtenez `BucketAlreadyExists`, alors qu'aucun bucket de ce nom n'existe dans votre compte ni dans votre région. Comment est-ce possible ?

Question 2 — S3 Standard affiche une durabilité de 99,999999999 % (11 neufs). Qu'est-ce que ce chiffre garantit exactement — et citez deux choses qu'il ne garantit **pas**.

Quiz — questions 3 et 4

Question 3 — Vous modifiez 2 fichiers sur les 40 de votre front local. Quelle commande n'enverra vers le bucket **que** ces 2 fichiers, et de quel outil du bloc Linux est-elle l'équivalent ?

Question 4 — Vous devez archiver des sauvegardes consultées au plus une fois par an, avec un délai de restitution de 12 h acceptable. Quelle classe de stockage choisissez-vous, et quels sont les deux pièges tarifaires à connaître ?

Quiz — questions 5 et 6

Question 5 — Le versioning d'un bucket peut-il être désactivé après activation ? Et que se passe-t-il exactement lorsqu'on exécute `aws s3 rm` sur un objet d'un bucket versionné ?

Question 6 — Le Block Public Access est activé **au niveau du compte**. Vous posez sur un bucket une bucket policy de lecture publique parfaitement valide. Le site est-il accessible ? Justifiez.

Quiz — questions 7 et 8

Question 7 — Quel champ est obligatoire dans une politique de bucket mais n'apparaît jamais dans une politique IAM attachée à un utilisateur ? Que désigne-t-il, et pourquoi cette différence ?

Question 8 — La politique IAM d'alice contient un **Deny** explicite sur `s3:GetObject`. La bucket policy du bucket visé contient un **Allow** pour alice sur cette même action. Que se passe-t-il quand alice tente de lire l'objet ?

🗨 Quiz — questions 9 et 10

Question 9 — Vous générez une URL présignée avec `--expires-in 300`. De qui cette URL tient-elle ses permissions, et que se passe-t-il à la 301^e seconde ?

Question 10 — Sur un rôle IAM : quelle est la différence entre la **trust policy** et la **permissions policy** ? Et pourquoi préfère-t-on attacher un rôle à une EC2 plutôt que d'y configurer des clés d'accès ?

À demain !

Votre front StockLine est en ligne, vos machines ont une identité sans clés. La suite du bloc :

- **J5 — RDS** : la base PostgreSQL de StockLine devient **managée** — sauvegardes, patches et haute dispo sans `apt install postgresql`.
- **J6 — ELB + Auto Scaling** : la haute disponibilité — plusieurs instances derrière un répartiteur de charge, qui naissent et meurent toutes seules.
- **J7 — Route 53, CloudFront, ACM** : un vrai nom de domaine, un CDN devant votre bucket S3 et **le HTTPS qui manque à notre site statique**.
- **J8 — CloudWatch, facturation, boto3** : superviser, comprendre la facture, et piloter AWS depuis Python.

Automatiser, superviser, sécuriser. 🙌 Ce soir : vérifiez votre teardown — `aws s3 ls` et la console EC2 ne doivent montrer que ce que vous assumez de payer.